

Manual Completo de Desarrollo e Interfaz de Usuario: TecnoNova Store

Plataforma de Comercio Electrónico con Arquitectura Limpia, MySQL y Stripe

Este documento representa el manual técnico y operativo definitivo de la plataforma **TecnoNova Store**. Aquí se documentan todos los pasos de desarrollo, las decisiones arquitectónicas bajo el patrón de **Clean Architecture**, y los flujos detallados de experiencia del cliente y del administrador, acompañados por capturas de pantalla de la interfaz de usuario.

1. Arquitectura de Software: Clean Architecture

El proyecto se estructuró dividiendo las responsabilidades en capas concéntricas, donde las dependencias fluyen únicamente hacia el interior. Esto aísla por completo las reglas de negocio de los detalles técnicos del entorno.

Estructura del Código Fuente

- **Capa de Dominio (Domain):** Define el núcleo del negocio.
 - **User.ts** ([User.ts](file:///C:/Users/jaira/Documents/tienda-api/src/domain/entities/User.ts)): Entidad de usuario (id, email, password, name, role).
 - **Product.ts** ([Product.ts](file:///C:/Users/jaira/Documents/tienda-api/src/domain/entities/Product.ts)): Entidad de producto (id, name, description, price, stock).
 - **Order.ts** ([Order.ts](file:///C:/Users/jaira/Documents/tienda-api/src/domain/entities/Order.ts)): Entidad de pedido (id, total, status, createdAt, userId, items).
 - **Capa de Aplicación (Application):** Ejecuta los casos de uso específicos.
 - **ManageOrders.ts** ([ManageOrders.ts](file:///C:/Users/jaira/Documents/tienda-api/src/application/use-cases/ManageOrders.ts)): Lógica de creación de órdenes y filtrado de pedidos por usuario.
 - **ManageProducts.ts** ([ManageProducts.ts](file:///C:/Users/jaira/Documents/tienda-api/src/application/use-cases/ManageProducts.ts)): Lógica para consultar catálogo, agregar o eliminar productos del stock.
 - **Capa de Infraestructura (Infrastructure):** Conexiones externas.
 - **mysql.ts** ([mysql.ts](file:///C:/Users/jaira/Documents/tienda-api/src/infrastructure/database/mysql.ts)): Pool de conexiones y migraciones de tablas a Aiven MySQL.
 - **server.ts** ([server.ts](file:///C:/Users/jaira/Documents/tienda-api/src/infrastructure/web/server.ts)): Inicialización de Express y enrutamiento seguro de peticiones HTTP.
-

2. Flujo de Usuario Paso a Paso (Con Imágenes)

Paso 1: Pantalla de Entrada de Login y Registro

Cuando un cliente normal ingresa al dominio público de la tienda, el sistema detecta que no tiene sesión iniciada. Se ocultan de inmediato el catálogo, el carrito y cualquier otra sección del sitio, presentándole en su lugar una pantalla limpia de autenticación en formato de pestaña individual:



Aquí, el cliente puede:

1. Digitar su correo electrónico y contraseña registrados.
2. Hacer clic en **"Regístrate aquí"** para cambiar al formulario de registro en la misma pantalla.
3. Una vez autenticado exitosamente, el backend genera un token de seguridad JWT que el navegador almacena de forma local para dar paso a la tienda.

Paso 2: Navegación del Catálogo de Productos

Al ingresar con un usuario válido, el catálogo se desbloquea. La interfaz se ajusta para mostrar el menú de navegación completo, el saludo con el nombre del usuario logueado en la esquina superior derecha, y la grilla de productos disponibles con sus especificaciones de stock y precios:



- Los productos con stock disponible muestran el botón dinámico **"Añadir al Carrito"**.
- Si el stock llega a 0, el botón se deshabilita automáticamente y muestra el indicador **"Agotado"**.

Paso 3: Gestión del Carrito de Compras

Cuando el usuario añade productos, se actualiza el contador en la cabecera. Al hacer clic en el carrito, se despliega un panel lateral deslizante (Drawer) que le permite:

- Ver el resumen de su compra (nombres, cantidades seleccionadas y costo).
- Visualizar el total a pagar calculado en tiempo real.
- Presionar el botón **"Proceder al Pago"** para iniciar la transacción segura.



Paso 4: Pasarela de Pago Segura (Stripe Checkout)

Una vez que el usuario avanza al pago, el servidor genera una sesión encriptada en los servidores de Stripe y redirige al cliente al formulario de cobro seguro de Stripe, asociando la transacción al identificador (**userId**) del cliente mediante metadatos cifrados:



Tras ingresar la tarjeta de prueba con éxito:

1. Stripe procesa el pago.
2. Redirige al usuario a la página de éxito de la tienda.
3. Stripe envía una solicitud POST firmada criptográficamente (Webhook) a la ruta **/api/payments/webhook** del servidor en Render.

4. El servidor procesa el Webhook, descarga el inventario de los productos comprados en MySQL e inserta el pedido con estado **PAID** relacionado a la cuenta del comprador.

Paso 5: Panel del Servidor (Administración General)

Cuando tú (el administrador del servidor) entras a la plataforma mediante tu URL segura (`?admin=tecnonova-admin`), el sistema se salta automáticamente el formulario de login, te autentica con el JWT del administrador y te muestra la interfaz completa de gestión:

 Panel de Administración General

En este panel dispones de tres secciones estratégicas en una grilla responsiva corregida contra desbordamientos:

1. **Añadir Nuevo Producto:** Formulario interactivo para ingresar el ID, nombre, descripción, precio y stock de nuevos productos para el catálogo general.
2. **Inventario Actual:** Listado de todos los artículos de la tienda con opción de eliminarlos con un clic.
3. **Usuarios Registrados y Ventas:** Tabla de auditoría que lista a todos los clientes registrados en la base de datos MySQL (nombre, correo y rol).

Paso 6: Auditoría de Historial de Compras por Usuario

Dentro de la tabla de usuarios, al hacer clic en el botón **"Ver Compras"** en la fila de cualquier usuario, la aplicación se comunica con el servidor para extraer únicamente los pedidos de dicho usuario y desplegarlos en una ventana modal interactiva:

```
// Llamada para obtener las compras de un usuario específico
async function showUserOrders(userId, userName) {
  try {
    userOrdersModalTitle.innerText = `Pedidos de ${userName}`;
    userOrdersContainer.innerHTML = '<p>Cargando compras del usuario...</p>';
    toggleUserOrdersModal(true);

    const res = await fetch(`/api/orders?userId=${userId}`);
    const orders = await res.json();
    renderUserOrders(orders);
  } catch (err) {
    showToast('Error al cargar pedidos.');
```

Esto te permite auditar de forma ágil y en tiempo real el comportamiento y transacciones individuales de cada uno de tus clientes de manera centralizada.

3. Código Fuente del Backend (Fragmentos Esenciales)

1. Endpoint seguro para listar usuarios (`authRoutes.ts`)

```
router.get('/users', authMiddleware, async (req: AuthenticatedRequest, res: Response) => {
  try {
    if (req.user?.role !== 'admin') {
      return res.status(403).json({ error: 'Acceso denegado. Rol insuficiente.' });
    }
    const rows = await queryAll<{ id: string; email: string; name: string; role: string }>(
      'SELECT id, email, name, role FROM users ORDER BY name ASC'
    );
    res.json(rows);
  } catch (error: any) {
    res.status(500).json({ error: error.message || 'Error al obtener usuarios.' });
  }
});
```

2. Filtrado de órdenes por query parameter (`orderRoutes.ts`)

```
router.get('/', authMiddleware, async (req: AuthenticatedRequest, res: Response) => {
  try {
    const user = req.user;
    if (!user) {
      return res.status(401).json({ error: 'Usuario no identificado.' });
    }

    // Si es admin, ve todas las órdenes por defecto. Si se pasa un userId, filtra por ese usuario.
    let targetUserId = user.role === 'admin' ? undefined : user.id;
    if (user.role === 'admin' && req.query.userId) {
      targetUserId = String(req.query.userId);
    }

    const orders = await manageOrders.getOrders(targetUserId);
    res.json(orders);
  } catch (error: any) {
    res.status(500).json({ error: error.message });
  }
});
```



4. Despliegue en Producción (Render + Aiven)

Para desplegar esta aplicación de forma óptima en Render, se configuraron las siguientes variables de entorno:

Variable	Tipo	Descripción
DB_HOST	MySQL Connection	Endpoint de conexión de tu base de datos de Aiven.
DB_PORT	MySQL Port	Puerto de conexión de la base de datos (por defecto 25887).
DB_USER	MySQL Credentials	Usuario de la base de datos (ej. <code>avnadmin</code>).
DB_PASSWORD	MySQL Credentials	Contraseña proporcionada en la consola de Aiven.
DB_NAME	MySQL Database	Base de datos activa (ej. <code>defaultdb</code>).
JWT_SECRET	Cryptography	Clave privada para firmar y validar tokens de sesión.
STRIPE_SECRET_KEY	Payment Gateway	Clave secreta para interactuar con la API de Stripe Checkout.
STRIPE_WEBHOOK_SECRET	Webhook Validation	Clave <code>whsec_...</code> obtenida al activar el webhook en Stripe.

[!NOTE] Toda la base de datos y los flujos descritos están activos en tu servidor en producción. Las contraseñas de tus usuarios nunca viajan ni se guardan en texto plano en la base de datos, garantizando los estándares modernos de seguridad web (OWASP).